Experimental Characterization of LoRa Propagation and Machine Learning Approaches for Path Loss Modeling Across Various Hill Terrain Types in Nepal

Peter Karki
ST7072CEM Machine Learning
Softwarica College of IT and E-Commerce
Coventry ID: 17624929
260054@softwarica.edu.np

***Abstract*—LoRa could support sensing in Nepal's hills, but most path loss models come from flat or urban areas. We built our own two-node 866 MHz instrument with a fixed Station and a walking Roamer. At each stop, the Station sends a full sweep of six spreading factors (SF7–SF12) at three power levels (2, 8 and 14 dBm) with five packets each, so every place gives a small experiment rather than one measurement. Across three cases near Kathmandu, we collected 61 complete stops (5,490 packet slots). All packets from one stop always stayed together during training and testing, and whole cases were also held out. A depth-3 decision tree using only spreading factor and power predicted delivery with a balanced accuracy of 0.68, and 0.67 on an unseen case. Path loss was not predictable from distance or endpoint terrain features: no model beat the mean-only baseline (RMSE 11.1 dB), and a log-distance fit held only in the short-range case ($n = 2.10$). A short sweep at each site found settings that saved a median 97% of transmit energy compared with SF12 at 14 dBm. An interactive dashboard maps predicted dead zones on example terrain. In hill terrain, what lies between the antennas matters more than the distance between them.**

***Index Terms*—LoRa, path loss, machine learning, hill terrain, grouped cross-validation, Nepal**

I. Introduction

LoRa is a low-power, long-range radio that is frequently used for Internet of Things (IoT) sensing. It could help with agriculture and hazard monitoring in areas of Nepal's hills where cellphone coverage is inconsistent. An estimate of path loss, often derived from the log-distance model in which loss increases with the logarithm of distance, is required for the planning of such networks [1]. Its parameters, however, come mostly from flat or urban areas. While locations kilometres apart across a valley may be able to view each other clearly, two places a few hundred metres apart may be divided by a ridge. As a result, planners must rely on trial and error, and Nepal lacks field data from such terrain.

Although machine learning (ML) can incorporate topography information into path loss prediction, the extent to which published results can be used is limited by two issues: random train–test splits might overstate accuracy by placing measurements from the same location in both sets, and few studies address hill terrain in South Asia. Therefore, we developed a device that measures three different case studies and tests every possible combination of six spreading factors and three transmit powers at each stop, separating the impact of radio settings from that of location (Table I). The research questions are:

TABLE I: The three case studies

Case	Site	Setting	Range	Stops
1	A	Forested slope; Station antenna 2.0 m	22–250 m	17
2	B	Hill slope across a valley; antenna 1.0 m	0.2–2.9 km	36
3	C, D	Town rooftops (≈9 and 6 m)	0.3–3.3 km	8

- **RQ1:** Do distance and simple terrain features improve path loss prediction over log-distance or mean-only baselines?
- **RQ2:** Can models that have been trained in one terrain case be used in another?
- **RQ3:** What is the cheapest radio setting each location needs?

The contributions are:

1) a reproducible two-node instrument that records delivery, RSSI and SNR for 18 radio configurations at every stop, and a dataset of 61 complete sweeps in three hill-terrain cases;
2) an evaluation of regression, decision-tree and ensemble models against simple baselines, using validation grouped by stop and a leave-one-case-out test;
3) evidence that radio settings transfer between terrains while distance-based path loss does not, an explanation of why, and the cheapest working setting at each stop;
4) an interactive dashboard that presents the results and maps predicted dead zones on example terrain.

Table II lists the abbreviations used.

TABLE II: Abbreviations

Abbreviation	Full form
AP / AUC	Average precision / area under the ROC curve
IoT	Internet of Things
GPS	Global Positioning System
LiDAR	Light detection and ranging
LOCO / LOO	Leave-one-case-out / leave-one-stop-out
OLS	Ordinary least squares
LoRa	Long Range (radio modulation)
LPWAN	Low-power wide-area network
ML	Machine learning
NOACK	No acknowledgement (handshake failed)
PL	Path loss
RF	Random forest
RMSE	Root-mean-square error
ROC	Receiver operating characteristic
RQ	Research question
RSSI / SNR	Received signal strength / signal-to-noise ratio
SF	Spreading factor
SRTM	Shuttle Radar Topography Mission

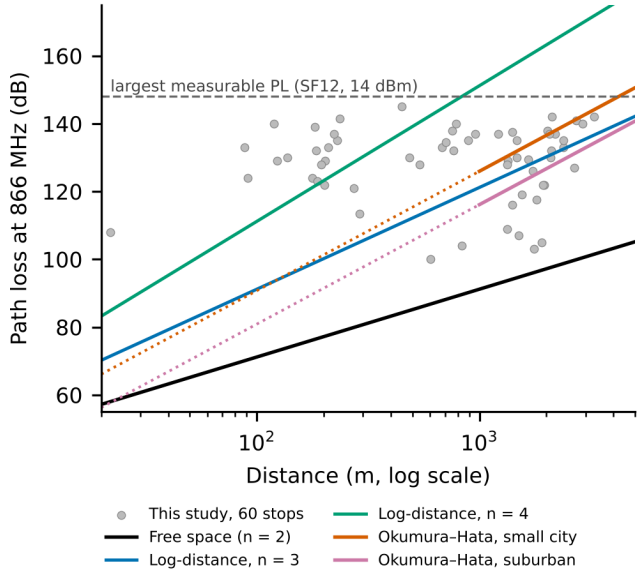


Fig. 1: Classical path loss models at 866 MHz with this study's 60 stops (grey). Drawn by the author from the published equations.

II. Literature Review

A. Classical path loss models

A path loss exponent n and a shadowing term are fitted to observations by the log-distance model [1]. The Okumura–Hata model adds corrections for frequency and antenna heights but assumes base stations above 30 m and distances over 1 km [4], while diffraction models such as ITU-R P.526 estimate the additional loss across hills [5]. At 866 MHz these models differ by more than 40 dB at 1 km, and none follows the spread of our stops (Fig. 1).

B. Empirical LoRa measurements

Petäjäjärvi *et al.* [2] measured LoRa at 868 MHz in Oulu, Finland, and reported ranges over 15 km on flat ground and about 30 km over water at SF12 and 14 dBm (Fig. 2). They also derived a channel attenuation model for similar flat areas.

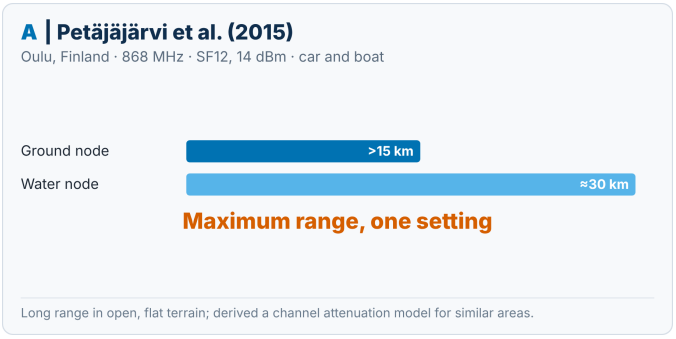


Fig. 2: Conceptual summary of [2]: maximum range with one setting in flat terrain. Drawn by the author.

In a 200 km^2 urban region, Rademacher *et al.* [3] gathered more than 112,000 samples over the course of 230 days and compared eleven models. While terrain-based models provided no gain, log-distance, Okumura and Winner+ provided fair predictions (Fig. 3).



Fig. 3: Conceptual summary of [3]: eleven empirical models compared in a city. Drawn by the author.

Bianco *et al.* [6] measured 868 MHz LoRa in an Alpine canyon using a single setting (SF7, 14 dBm) and only reached roughly 300 m at 50% delivery, which is significantly less than on level ground (Fig. 4).

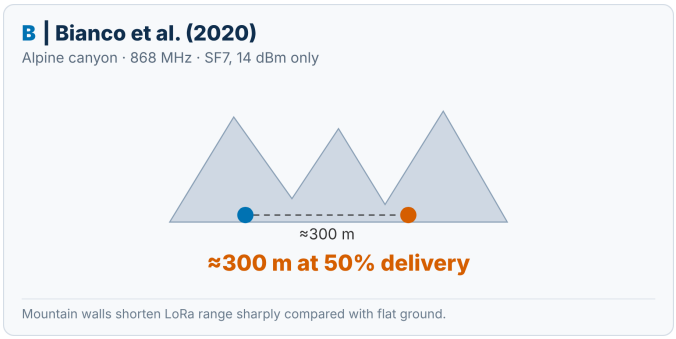


Fig. 4: Conceptual summary of [6]: range at 50% delivery in an Alpine canyon. Drawn by the author.

Pokharel *et al.* [8] suggested settings from a single, roughly 5 km 433 MHz link in Nepal without ML (Fig. 5). All four studies used one link or one radio setting at a time, which motivates multi-configuration measurements at many locations.

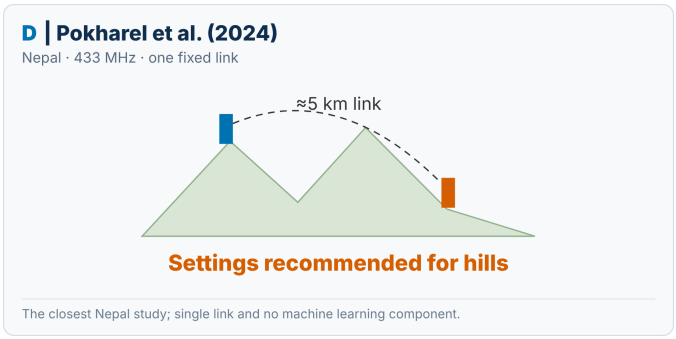


Fig. 5: Conceptual summary of [8]: one long link in Nepal used to recommend settings. Drawn by the author.

C. Machine learning for path loss

Ojo *et al.* [12] fed the sequence of land-cover classes along each link to a bidirectional LSTM and reported a mean error of 4.97 dB from over 35,000 packets of sheep-tracking devices on Alpine pastures, about half the error of established empirical models (Fig. 6).

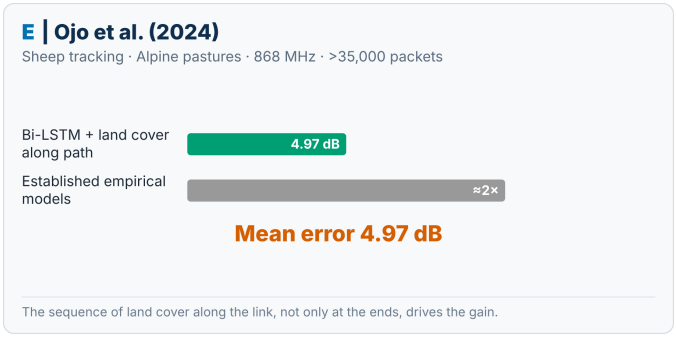


Fig. 6: Conceptual summary of [12]: a sequence model using land cover along the path. Drawn by the author.

Adding obstacle depth along the path reduced the median RMSE by around 5 dB, to 7.5 dB, as demonstrated by Ethier and Châteauvert [7] using 8.2 million observations (Fig. 7).

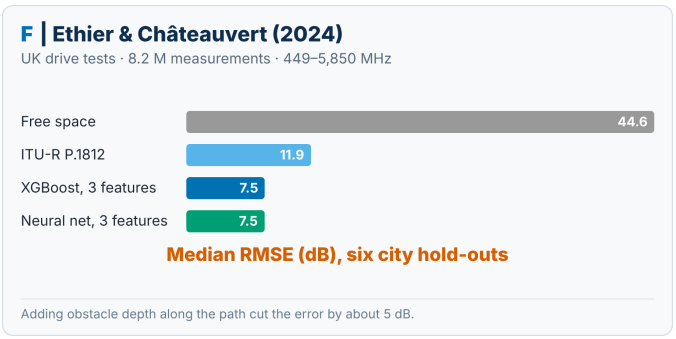


Fig. 7: Median RMSE across six held-out cities, conceptual overview of [7]. Drawn by the author.

Random forests [9] with LiDAR terrain features outperformed baselines under random splits, according to Bitterling *et al.* [10], but deteriorated when entire gateways were held out (Fig. 8). When taken as a whole, these studies imply that stated accuracy is significantly impacted by validation design and that the terrain *along the path* is relevant. Because their rules can be read and verified, decision trees [11] continue to be appealing for small datasets.

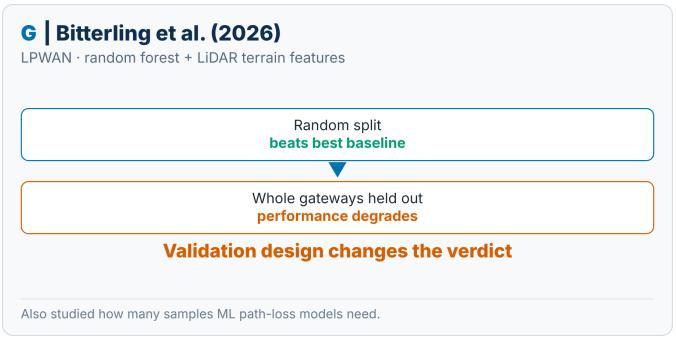


Fig. 8: Conceptual summary of [10]: the verdict changes with the validation design. Drawn by the author.

D. Censoring and validation

Ignoring weak links biases the fitted exponent since they are lost rather than measured; this is corrected by a Tobit estimator [13], [14] (Fig. 9). Samples grouped in space also make random cross-validation optimistic [15]; packets from one stop form such a group. Both problems have a significant impact on LoRa data since each site produces a large number of packets and many packets are lost at long range.

E. Why the reported metrics are not benchmarks

The studies answer different questions with different data: maximum range, delivery at one setting, mean error of a sequence model, or RMSE over held-out cities. Their numbers describe the field but cannot be compared to this study or to each other because of differences in their frequencies, sample

sizes, targets and validation techniques. As a result, Table III maintains each statistic in its original format.

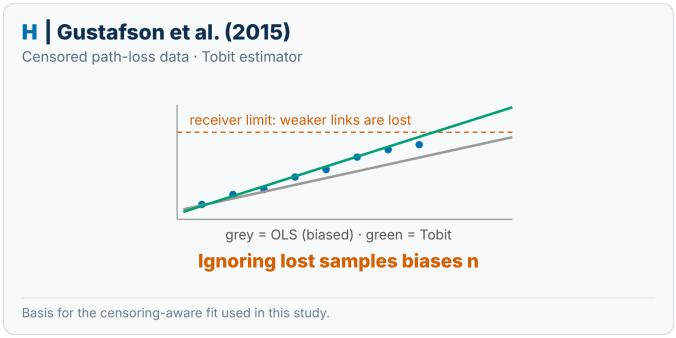


Fig. 9: Conceptual summary of [14]: ignoring lost samples biases the fitted exponent. Drawn by the author.

F. Research gap

No reviewed study combines multi-setting LoRa measurements in Himalayan foothills, explicit handling of censored and failed links, and ML validation that holds out whole terrain cases (Table IV). Due to its small size, this study checks whether the reviewed work's methods hold up in a different context rather than trying to outperform it.

III. Measurement Setup and Data

A. LoRa background

LoRa uses chirp spread spectrum [16], [17]. Each step from SF7 to SF12 doubles airtime, and the SNR required for decoding decreases by roughly 2.5 dB with each step, reaching approximately -20 dB at SF12. Thus, a greater SF allows links to endure below the thermal noise floor by exchanging airtime and energy for range. Each extra decibel of transmit power adds one decibel at the receiver. A link's margin can be determined by testing every SF–power pair in one location.

B. Instrument

Both nodes use an ESP32 and an Ai-Thinker Ra-08H module at 866.1 MHz and 125 kHz, which are within the spectrum that Nepal designates for IoT [18] (Figs. 10 and 11). The fixed *Station* transmits; the walking *Roamer* receives and logs every slot with RSSI, SNR and GPS position (5–10 m accuracy). Since the vendor firmware cannot directly alter SF and power, custom radio firmware was created. It was frozen for all measurements and fingerprinted using SHA-256 so that any changes would be detected. The Roamer firmware (version 4.6.0) logs every slot to a CSV file on a microSD card. Cable losses and antenna gains were not quantified.

C. Sweep protocol

At each stop the Roamer requests a sweep over a control link (SF12, 14 dBm). Next, in roughly 67 s, the Station sends five packets for each of the 18 configurations, totalling 90 slots, each of which is recorded as OK or TIMEOUT (Table V, Fig. 12). If the handshake fails (NOACK), no sweep runs, and the place is recorded as a failed attempt. Sweeps following strong handshakes began 3.5–7 s later in the Case 1 logs, but sweeps following weak handshakes began 11–46 s later. This is consistent with link margin rather than a clock error.

D. Data and cleaning

The raw data are one Roamer log per site. They were combined, and only complete sweeps were kept: 61 stops (17, 36 and 8 per case), 5,490 slots and 2,612 received packets (47.6%). The 56 failed handshakes reduce to 30 unique *lost locations* (3, 9 and 18 per case) after removing retries within 30 m of a completed stop. Due to significant differences in delivery at the same locations, an earlier Site A run on older firmware was eliminated. Every reported number is replicated by a scripted pipeline that reconstructs the stop table (61 rows) and packet table (5,490 rows) from the raw logs.

IV. Methodology

A. Workflow and problem formulation

Fig. 13 summarises the workflow. For stop s and slot k with configuration $c_k = (\mathrm{SF}_k, P_k)$, the targets are

$$y_s = \mathrm{median}\{PL_{s,k} : P_k = 14\,\mathrm{dBm},\ z_{s,k} = 1\}, \quad (1)$$

$$z_{s,k} = 1 \text{ if slot } k \text{ was received, else } 0, \quad (2)$$

$$c_s^* = \arg\min_c \ \mathrm{cost}(c) \ \text{s.t.} \sum_{k:c_k=c} z_{s,k} \geq 4, \quad (3)$$

with $PL = P_{tx} - \mathrm{RSSI}$ and $\mathrm{cost}(c) = 10^{P/10} \cdot 2^{\mathrm{SF}-7}$. Path loss y_s is a regression target, delivery $z_{s,k}$ a classification target and c_s^* the cheapest working setting (RQ3). Apparent path loss includes the unmeasured antenna gains. Only 14 dBm packets define y_s because the measured power steps were larger than nominal; 60 stops have a value, as one Case 1 stop received nothing.

B. Features

The haversine formula was used to calculate the distance between the Station and the stop. The ground elevations of stops and Stations were taken from the SRTM 30 m elevation model [19], which gave the elevation difference (stop minus Station) and the elevation angle between the antennas. Satellite imagery was used to identify the land cover surrounding each stop as forest, built-up, mixed or open (14, 12, 31 and 4 stops). Forest was then used as the reference for one-hot encoding. The models use $\log_{10}$ distance. Station antenna height (one value per site) and a manual line-of-sight label (made with delivery visible) were excluded to avoid leakage.

C. Models and why they were chosen

Since path loss is continuous, it is a regression problem. The models were a mean-only baseline; the log-distance model $PL(d) = PL_0 + 10\,n \log_{10} d + X_\sigma$; least squares and ridge regression with terrain features (ridge suits few, correlated features); a depth-3 regression tree; and a 300-tree random forest. When a Tobit fit is applied to all packets, every lost packet is treated as path loss above its measurable limit [13], [14].

TABLE III: Comparable studies (metrics in original form; not directly comparable)

Study	Setting	Method	Result	Limitation here
Petäjäjärvi [2]	Flat land and sea	SF12, 14 dBm	>15 km, ≈30 km	Flat; one setting
Rademacher [3]	Urban, 112k samples	11 empirical models	Log-distance adequate	City; no ML
Bianco [6]	Alpine canyon	SF7, 14 dBm	≈300 m at 50%	One setting
Pokharel [8]	Nepal, 433 MHz	One link	Settings advice	Single link
Ojo [12]	Alpine pastures	Bi-LSTM, land cover	4.97 dB mean error	Large dataset
Ethier [7]	UK drive tests	ML, obstacle depth	RMSE ≈7.5 dB	Urban, 8.2M samples
Bitterling [10]	LPWAN gateways	RF + LiDAR	Drops on new gateways	–
This work	Nepal hills, 61 stops	Trees, ridge, Tobit, RF	0.67 on unseen case	–

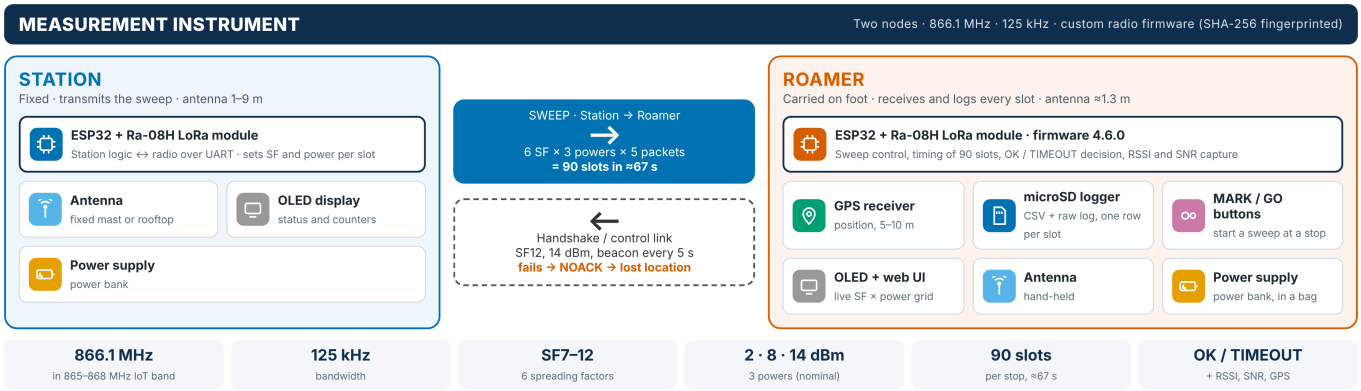
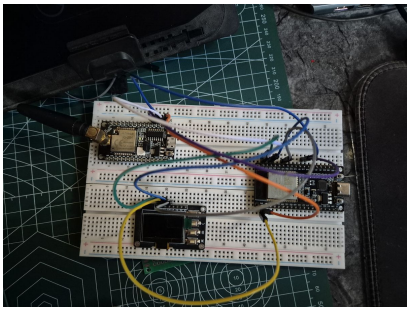
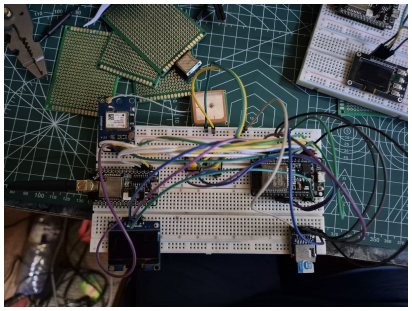


Fig. 10: Hardware of the two-node instrument and its two radio links.



(a) Station



(b) Roamer



(c) Roamer in the field

Fig. 11: The instrument and its use in the field.

TABLE IV: Research gaps

Gap	Prior work	This study
Terrain	Flat, urban, Alpine [2], [3], [6]	Three Nepal hill cases
Settings	One per campaign [6]	18 per stop
Censoring	Rarely handled [14]	Tobit; 30 lost locations
Validation	Random splits [10]	By stop and by case

TABLE V: Sweep configuration

Parameter	Value
Frequency / bandwidth	866.1 MHz / 125 kHz
Spreading factor	SF7–SF12
Transmit power	2, 8, 14 dBm
Packets per setting	5 (90 slots per stop)
Control link	SF12, 14 dBm

Delivery is a classification problem since it is either yes or no. Depth-3 trees [11] with at least 90 packets per leaf were the main classifier: receiver sensitivity behaves like a threshold, the rules are readable, no leaf can memorise one stop, and the model is small enough for a node. Baselines were the majority class and a link-budget rule, which predicts delivery when the transmit power minus the log-distance prediction exceeds the sensitivity of the spreading factor; a random forest [9], logistic regression and gradient boosting tested whether more flexible models help. Its random-split score was also recorded to show the effect of leakage. Neural networks were not used: with 61 independent locations, extra capacity would mainly memorise stops, and the sequence models that work well [12] relied on tens of thousands of links.

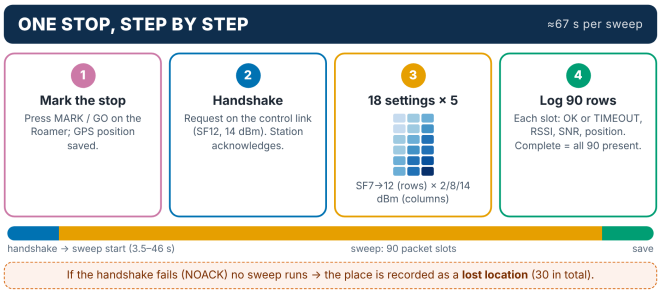


Fig. 12: One stop, step by step.

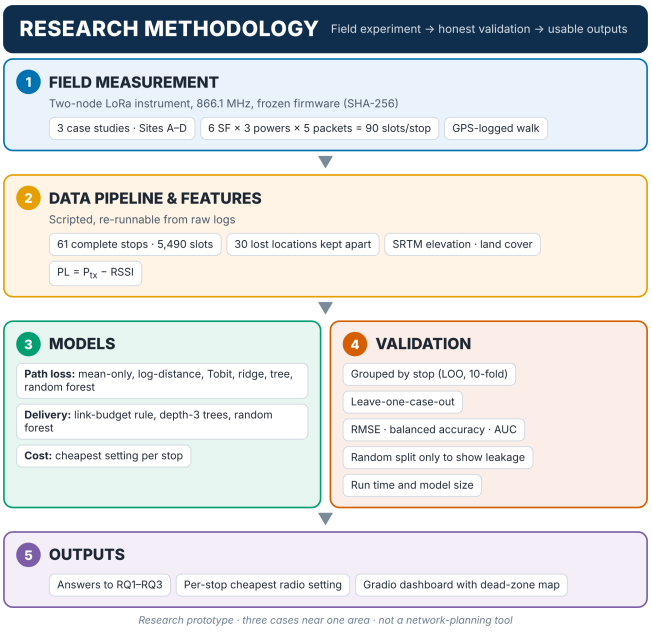


Fig. 13: Research methodology workflow.

D. Validation and metrics

Packets from a single place are not independent since they share topography and location [15]. If one stop's packets appeared in both training and test sets, a model could recognise the stop instead of learning a general relationship. Thus, each split keeps the packets of a stop together (Fig. 14). Classifiers employed 10-fold cross-validation grouped by stop and leave-one-case-out (LOCO), while regression employed leave-one-stop-out (LOO) and LOCO. Metrics were RMSE, balanced accuracy, precision, recall, F_1, AUC and average precision:

$$\mathrm{RMSE} = \sqrt{\tfrac{1}{N}\textstyle\sum_i (y_i - \hat{y}_i)^2}, \quad F_1 = \tfrac{2PR}{P+R}, \tag{4}$$

$$\mathrm{BalAcc} = \tfrac{1}{2}\left(\tfrac{TP}{TP+FN} + \tfrac{TN}{TN+FP}\right), \tag{5}$$

where $P = TP/(TP+FP)$ and $R = TP/(TP+FN)$. Balanced accuracy is the headline metric because it weighs lost and delivered packets equally (0.5 is chance). With three cases, LOCO results are indicative. Associations used Spearman's ρ and differences between cases the Kruskal–Wallis test.

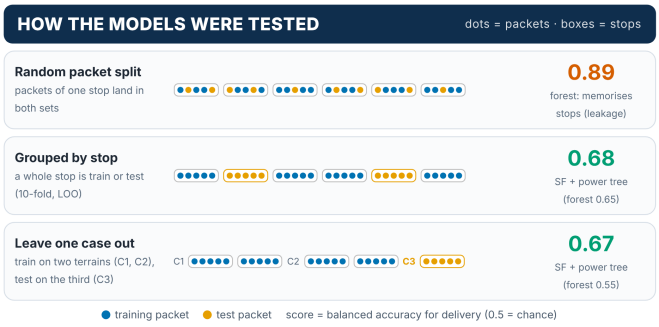


Fig. 14: Validation schemes and resulting balanced accuracy.

TABLE VI: Model settings

Model	Setting
Ridge	30 penalties, 10^{-2}–10^{3}
Regression tree	Depth 3, $\geq$5 stops per leaf
Delivery tree	Depth 3, $\geq$90 packets per leaf
Random forest	300 trees
Tobit	L-BFGS-B; 300 bootstrap resamples
Threshold / seed	0.5 / 0

Table VI lists the settings; the analysis used Python and scikit-learn [20].

E. Computational complexity

Least squares costs $O(nd^2)$ for n samples and d features, a tree costs $O(d\,n\log n)$ to train and $O(\text{depth})$ to predict, a forest of T trees costs T times more, and Tobit costs $O(kn)$ for k iterations. On a 2.1 GHz cloud computer, the medians of five single-thread runs were used to calculate training time, prediction time and model size. On a small node, a depth-3 tree responds with no more than three comparisons.

F. Dashboard

A Gradio dashboard (Appendix B) plots the tree's predicted dead zones on artificial hill terrain next to a geometric line-of-sight layer that the model is unable to perceive, displays the findings, and performs grouped validation in real time. The expected receiver sensitivity used for the instrument check is the thermal noise floor (≈ -117 dBm for 125 kHz and a 6 dB noise figure) plus the LoRa SNR limit of each spreading factor [16].

V. Results

A. Instrument and delivery

The weakest received packets matched the expected sensitivity: -124 dBm at SF7 (expected -124.5) and -134 dBm at SF12 (expected -137) (Fig. 15). The radios behaved as expected because the lowest SNRs (-10 to -21 dB) were within around 2.5 dB of the demodulation limits. Raising power from 2 to 14 dBm raised the median RSSI at the same stop and SF by 15.3 dB, not the nominal 12 dB (8 to 14 dBm: 8 dB; 2 to 8 dBm: 7 dB), which supports using only 14 dBm packets for path loss. In all cases, delivery increased with SF and power, rising from 16% at SF7/2 dBm to 88% at SF12/14 dBm

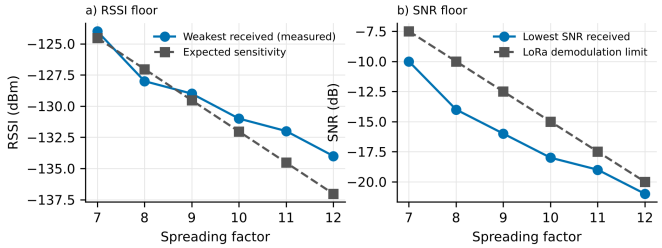


Fig. 15: Weakest received RSSI and lowest SNR per SF against expected limits.

TABLE VII: Case-study profiles

	Case 1	Case 2	Case 3
Stops (sites)	17 (A)	36 (B)	8 (C, D)
Median distance (m)	185	1,374	2,248
Distance range (m)	22–250	209–2,916	272–3,268
Elevation diff. (m)	0 to 36	−34 to 113	−4 to 287
Received / slots	563/1,530	1,737/3,240	312/720
Delivery (%)	36.8	53.6	43.3
Median PL_{14} (dB)	129.5	130.0	134.0
Lost locations	3	9	18
Link success (%)	85	80	31
ρ(delivery, distance)	−0.58*	−0.02	−0.42
ρ(delivery, elev. diff.)	−0.26	0.49*	−0.04

*$p < 0.05$.

(Fig. 16). At SF12/14 dBm, delivery was 75%, 92% and 95% in Cases 1–3. In contrast to Case 2, which delivered 24% at SF7/2 dBm, Case 1 delivered only 2%, indicating significant obstruction at close range. The majority of stops were neither completely connected nor disconnected, which is the regime where configuration decisions are most important. Differences between cases were not significant (Kruskal–Wallis $p = 0.18$ for delivery, $p = 0.34$ for path loss; Table VII).

B. Path loss, distance and link existence

Path loss was almost independent of distance when pooled over 60 stops ($n = 0.14$, 95% CI −0.44 to 0.72, $\sigma = 11.1$ dB; Fig. 17, Table VIII). Only Case 1 followed the expected trend ($n = 2.10$, $R^2 = 0.41$). Case 3's fit is based on eight stops with a confidence interval that includes zero, while Case 2's path loss at comparable distances ranged from roughly 100 to 145 dB and showed no pattern. The censoring-aware Tobit fit was also flat ($n = -0.12$, CI −1.12 to 0.64). Delivery rose with the stop's elevation above the Station ($\rho = 0.33$, $p = 0.008$), driven by Case 2 ($\rho = 0.49$), while delivery fell with distance only in Case 1 ($\rho = -0.58$). Link success decreased from 91% within 300 m to 52% at 2–4 km, and it was only 31% in Case 3 when the 30 lost locations were taken into account (Fig. 18).

C. Path loss regression

No model beat the mean-only baseline (Table IX, Fig. 19). Under LOCO, terrain features raised the error to 15.0–15.8 dB, including for the random forest, against 11.4 dB for the baseline and 13.0 dB for log-distance; the largest errors occurred when Case 3 was held out. Elevation was somewhat helpful

TABLE VIII: Log-distance fits of median 14 dBm path loss

Fit	PL_0 (dB)	n	95% CI of n	R^2	σ (dB)
All stops (60)	124.5	0.14	−0.44, 0.72	0.00	11.1
Case 1 (16)	84.3	2.10	0.65, 3.56	0.41	6.7
Case 2 (36)	128.8	−0.07	−1.66, 1.53	0.00	12.6
Case 3 (8)	88.0	1.41	−0.15, 2.97	0.45	5.8
Tobit, all packets	140.6	−0.12	−1.12, 0.64	–	17.7
Tobit, Case 1	73.2	3.09	–	–	12.4

TABLE IX: Path loss regression RMSE (dB)

Model	LOO	LOCO	LOCO per case
Mean only	**11.1**	**11.4**	8.4 / 12.9 / 9.1
Log-distance	11.3	13.0	9.8 / 14.8 / 9.5
+ elevation (OLS)	11.4	15.8	11.2 / 16.3 / 20.9
+ land cover (ridge)	11.1	15.7	8.4 / 17.1 / 19.8
Regression tree	11.2	15.3	9.5 / 16.9 / 16.8
Random forest	11.3	15.0	11.8 / 16.5 / 13.0

within one case: in Case 1, the LOO RMSE decreased from 8.7 to 7.2 dB. Restricting the data to 51 stops with at least ten received packets gave the same picture. The excluded manual line-of-sight label did separate delivery (27%, 53% and 76% for blocked, partly blocked and clear), which shows that path obstruction matters.

D. Delivery classification

A depth-3 tree on SF and power reached a balanced accuracy of 0.68 (AUC 0.71), and 0.67 under LOCO, holding at 0.69, 0.67 and 0.72 for the held-out Cases 1–3 (Table X). LOCO was reduced to 0.50 by adding terrain features (Fig. 20). The random forest scored 0.89 under a random packet split but 0.65 grouped and 0.55 on an unseen case: it recognised stops rather than learning transferable rules. The tree was the most precise model (0.70); the link-budget rule predicted almost every packet as delivered (recall 0.97, precision 0.50). Within known cases, gradient boosting performed slightly better (AUC 0.75), but on an unseen case it dropped to 0.58 (Fig. 21). The tree accurately identified 2,217 of the 2,878 lost packets and found 1,534 of the 2,612 delivered ones. The learning curve increased from 0.57 with nine training stops to 0.62–0.64 with 39–49 stops before levelling off. The tree fitted to all data splits first on power ($\leq$11 dBm): SF11–12 delivers the majority of packets at 14 dBm, and at low power, delivery is expected only when the stop is more than roughly 57 m above the Station (Fig. 22).

E. Cheapest setting and computational cost

At 55 of 61 stops, a setting delivered at least four out of five packets; 19 required only SF9 or below at 8 dBm or below, and 7 required SF11–12 at 14 dBm. The median stop could have used 97% less energy than SF12 at 14 dBm (92% in Case 3). These choices indicate what was measured rather than a guaranteed long-term setting because they are based on five packets per setting. The delivery tree trained in about 1–3 ms and occupied about 2 kB, whereas the forest took 0.86 s and about 24 MB without performing better (Table XI).

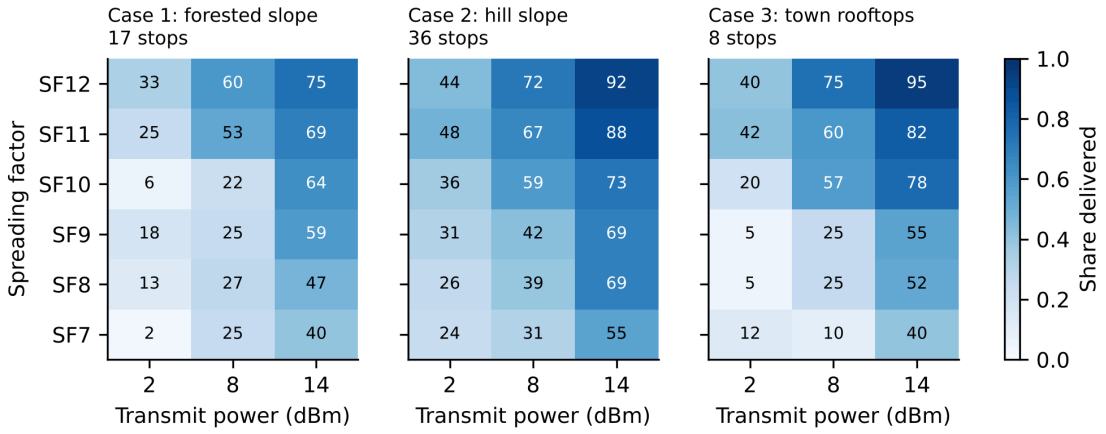


Fig. 16: Share of packets delivered for each SF and transmit power, per case.

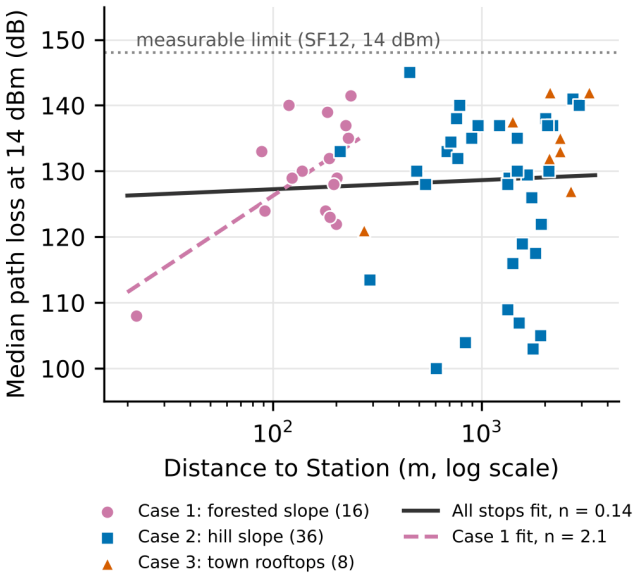


Fig. 17: Median 14 dBm path loss against distance with log-distance fits.

TABLE X: Delivery metrics, 10-fold grouped (threshold 0.5)

Model	BalAcc	Prec.	Rec.	F1	AUC	AP	LOCO
Link-budget rule	0.54	0.50	0.97	0.66	0.73	0.71	–
Tree, SF + power	**0.68**	**0.70**	0.59	0.64	0.71	0.68	**0.67**
Tree, all features	0.62	0.63	0.53	0.57	0.64	0.60	0.50
RF, all features	0.65	0.63	0.62	0.63	0.71	0.66	0.55
Logistic, SF + power	0.68	0.67	0.64	0.65	0.73	0.71	0.65
Logistic, all	0.67	0.66	0.64	0.65	0.73	0.70	0.56
Grad. boosting, all	0.68	0.66	0.67	**0.67**	**0.75**	**0.72**	0.58

VI. Discussion

A. Answers to the research questions

- **RQ1:** Distance and endpoint terrain features did not beat the mean-only baseline; distance mattered only in Case 1 and for whether a link formed at all.

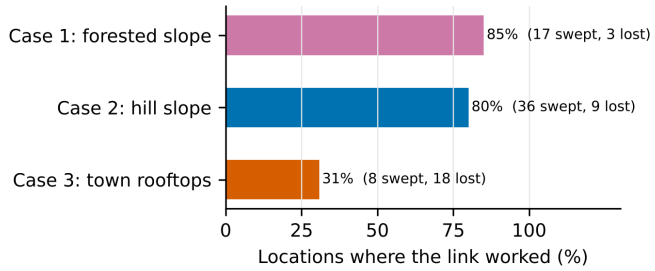


Fig. 18: Share of attempted locations where a sweep was completed.

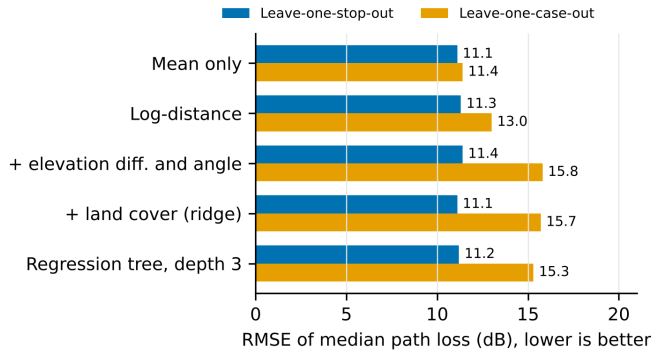


Fig. 19: Path loss RMSE under LOO and LOCO validation.

- **RQ2:** Radio-setting models transferred (0.67 on an unseen case); terrain-feature and more flexible models did not.
- **RQ3:** Most stops required significantly less energy than the safest setting, and the cheapest setting varied greatly between stops.

Fig. 23 summarises these findings.

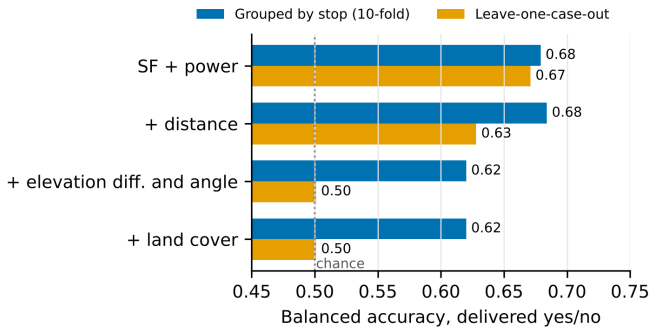


Fig. 20: Balanced accuracy of depth-3 trees for growing feature sets.

TABLE XI: Measured computational cost

Model	Complexity	Fit (ms)	Size (kB)
Log-distance (OLS)	$O(nd^2)$	0.6	0.5
Ridge	$O(a\,nd^2)$	2.2	1.2
Regression tree	$O(d\,n \log n)$	0.5	1.7
RF regressor	$O(Td\,n \log n)$	220	1,581
Tobit	$O(kn)$	28.9	0.2
Tree, SF + power	$O(d\,n \log n)$	1.2	2.3
RF classifier	$O(Td\,n \log n)$	865	24,405

B. Why distance and terrain features failed

Three effects flatten the pooled distance trend. Censoring eliminates the weakest packets, but the Tobit fit indicates that this is not the complete picture. Survivorship hides 30 mostly distant locations where no link formed. Including the lost locations, link success falls from 91% to 52% with distance, a trend the sweep data alone hide. Because of mixed terrain, distant stops across a valley can have a better view of the Station than nearby stops behind a ridge; Case 2's positive correlation between delivery and stop elevation supports this. Endpoint features characterise the two ends of a link but not the terrain between them, which is consistent with the gains reported from path features [7], [12] and the drop on held-out gateways [10].

C. Why features did not transfer

Terrain features were somewhat helpful within one case, but across cases they encoded case-specific correlations and increased the error, particularly for the small Case 3. A model that was trained where height above the Station predicted delivery (Case 2) misled it where distance was more important (Case 1). Bitterling *et al.* [10] observed the same failure when entire gateways were held out, which is why only small, interpretable models were employed.

D. Was the model choice justified?

Yes. Every flexible model lost accuracy on an unseen case, while both SF + power models kept it. The shallow tree matched the best transferable model, was the most precise, employed three readable rules and was around 10,000 times smaller than the forest. Regression was the appropriate framework for path loss, but no regression model—no matter how basic or sophisticated—beat the mean; the information in the features, not the model, was the limit. With 61 stops, simplicity was a strength, not a compromise.

E. Comparison and practical use

The shadowing levels (5.8–12.6 dB) are common for outdoor LoRa, while the Case 1 exponent (2.10) is close to free space. A setting suggested from one link [8] should not be applied elsewhere since, in contrast to flat-terrain ranges of tens of kilometres [2], 48% of hill locations beyond 2 km formed no link. The censoring-corrected Case 1 exponent (3.09) is more in line with values for obstructed links. The larger-sample models in the literature reach 4.97 to 7.5 dB [7], [12] because they describe the whole path. For practice, a planner without terrain data can expect most packets to arrive at SF11–12 and 14 dBm; however, at a typical stop, this safe setting wastes the majority of the energy. A short on-site sweep finds a much cheaper setting per node, path loss from distance alone should carry an uncertainty of about 11 dB, and the tiny tree could run on the node itself.

F. Ethics and reproducibility

No personal data were collected. Site names are coded as Sites A–D. The GPS positions of the stops and Stations are kept out of the paper text, but they are published in the raw logs so the analysis can be reproduced. Predictions and the dead-zone map are research outputs, not coverage guarantees for safety uses. The dashboard replicates the main classifier result, while the analysis notebook reconstructs each table and figure from the raw logs.

VII. Limitations and Future Work

A. Limitations

This is a small study: 61 complete stops, only eight in Case 3, from two rooftop Stations of different estimated height, all in one season. Since antenna gains were not measured, path loss is apparent rather than absolute, and rooftop heights were estimated. GPS accuracy is about 5–10 m, SRTM resolution of 30 m limits the elevation features, and the Station-side log was unavailable. The dead-zone map applies a model trained on 61 stops to invented terrain and is an illustration only. The findings are specific to this area and should not be generalised beyond the three cases.

B. Future work

More stops, cases and seasons would provide stronger cross-case validation. Station-side logging and measured antenna gains would give absolute path loss. Terrain-profile features computed along each path, such as obstruction depth [7] or knife-edge diffraction loss [5], should be tested with censoring-aware models and the same strict validation. Lastly, the sweep might develop into an adaptive process in which a node tries settings from the least expensive to the most costly, keeping the first that works.

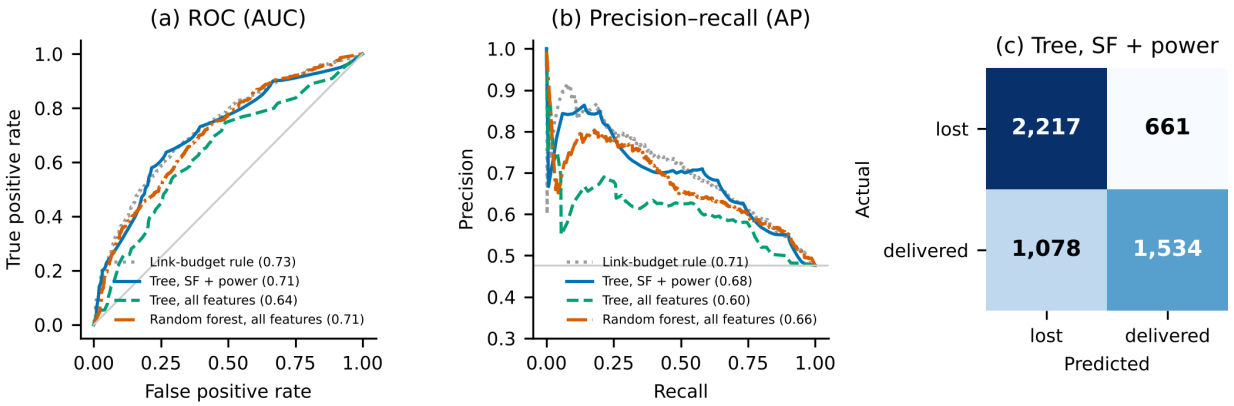


Fig. 21: (a) ROC and (b) precision–recall curves under grouped validation; (c) confusion matrix of the SF + power tree.

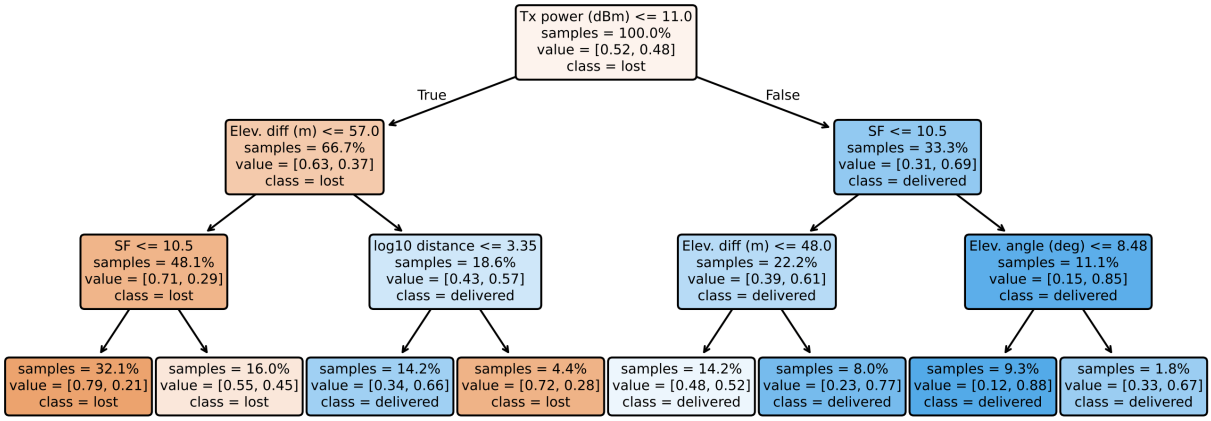


Fig. 22: Depth-3 delivery tree fitted to all data with all features (interpretation only).

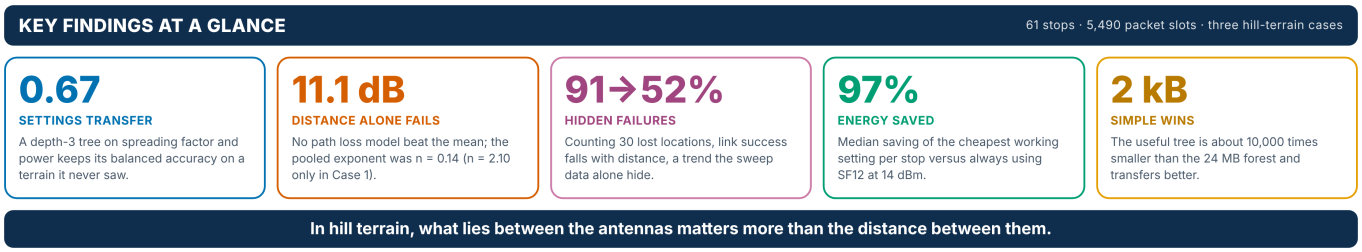


Fig. 23: Key findings at a glance.

VIII. Conclusion

Across 61 stops in three hill-terrain cases, delivery was governed by spreading factor and power, and a shallow tree on these two settings kept its accuracy on an unseen case while being among the cheapest models. More flexible models, including random forests and gradient boosting, did not transfer better. Path loss could not be predicted from distance or endpoint terrain better than by its mean (RMSE 11.1 dB), and a log-distance model held only in the short-range obstructed case. Per-stop adjustments could save a median of 97% of transmit energy. Receiver censoring, unrecorded failed links and terrain mixing explain much of this. In hill terrain, what lies between the antennas matters more than the distance between them.

Acknowledgment

I thank my module tutors at Softwarica College of IT and E-Commerce for their guidance and feedback. Code, figure layout and language editing were aided by generative AI tools; I was responsible for the experiment design, field measurements,

analysis choices and findings, and each reported number was verified against the analysis notebook.

Data Availability

Appendix C contains links to the dashboard code, stop features, analysis notebook and raw Roamer logs. The raw logs include GPS positions; site names are coded.

References

[1] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[2] J. Petäjäjärvi, K. Mikhaylov, A. Roivainen, T. Hänninen, and M. Pettissalo, "On the coverage of LPWANs: Range evaluation and channel attenuation model for LoRa technology," in *Proc. 14th Int. Conf. ITS Telecommunications (ITST)*, 2015, doi:10.1109/ITST.2015.7377400.

[3] M. Rademacher, H. Linka, T. Horstmann, and M. Henze, "Path loss in urban LoRa networks: A large-scale measurement study," *arXiv preprint arXiv:2109.07768*, 2021.

[4] M. Hata, "Empirical formula for propagation loss in land mobile radio services," *IEEE Trans. Veh. Technol.*, vol. 29, no. 3, pp. 317–325, 1980.

[5] ITU-R, "Propagation by diffraction," Recommendation ITU-R P.526, International Telecommunication Union, Geneva, Switzerland.

[6] G. M. Bianco, A. Mejia-Aguilar, and G. Marrocco, "Radio wave propagation of LoRa systems in mountains for search and rescue operations," in *Proc. URSI General Assembly and Scientific Symposium (URSI GASS)*, 2020.

[7] J. Ethier and M. Châteauvert, "Machine learning-based path loss modeling with simplified features," *arXiv preprint arXiv:2405.10006*, 2024.

[8] A. Pokharel, P. Sapkota, D. Sapkota, S. Dahal, S. Karki, and R. K. Budhathoki, "Performance evaluation of LoRa technology for rural connectivity: An experimental analysis in Nepal," *arXiv preprint arXiv:2412.04563*, 2024.

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi:10.1023/A:1010933404324.

[10] R. Bitterling, C. Nettersheim, J. Hees, and M. Rademacher, "A systematic sample size analysis of ML-based path loss prediction for LPWAN," *arXiv preprint arXiv:2608.11083*, 2026.

[11] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[12] M. O. Ojo, I. Viola, S. Miretti, E. Martignani, S. Giordano, and M. Baratta, "A deep learning approach for accurate path loss prediction in LoRaWAN livestock monitoring," *Sensors*, vol. 24, no. 10, p. 2991, 2024, doi:10.3390/s24102991.

[13] J. Tobin, "Estimation of relationships for limited dependent variables," *Econometrica*, vol. 26, no. 1, pp. 24–36, 1958.

[14] C. Gustafson, T. Abbas, D. Bolin, and F. Tufvesson, "Statistical modeling and estimation of censored pathloss data," *arXiv preprint arXiv:1504.03977*, 2015.

[15] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017, doi:10.1111/ecog.02881.

[16] Semtech Corporation, "LoRa modulation basics," Application Note AN1200.22, 2015.

[17] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A study of LoRa: Long range & low power networks for the Internet of Things," *Sensors*, vol. 16, no. 9, p. 1466, 2016, doi:10.3390/s16091466.

[18] Nepal Telecommunications Authority, "Radio Frequency Policy," Kathmandu, Nepal, 2023.

[19] T. G. Farr *et al.*, "The Shuttle Radar Topography Mission," *Rev. Geophys.*, vol. 45, no. 2, RG2004, 2007, doi:10.1029/2005RG000183.

[20] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

Appendix A
Field use and analysis notebook

Fig. 24 shows the Roamer carried during the field walk and its interface after a sweep, which displays delivery for each SF and power. Figs. 25–28 are excerpts of the executed analysis notebook. They cover data loading and checks, path loss regression, the delivery classifiers and the cost measurements. Cells printing GPS coordinates are omitted.

Appendix B
Interactive dashboard

Fig. 29 shows the Gradio dashboard in dark mode. Global filters for case, distance and land cover update every tab: overview, delivery explorer, path loss, settings advisor, model lab, dead-zone map and results. The settings advisor suggests the cheapest setting for a new node and shows the most similar measured stops as evidence. The SF + power tree result (balanced accuracy 0.68, AUC 0.71, precision 0.70, recall 0.59) is replicated in real time by the model lab. The dead-zone map is an illustration on synthetic terrain: where the tree and the line-of-sight layer disagree marks the information that endpoint features cannot capture.

Appendix C
Video and source code

Video demonstration: https://youtu.be/8JvQkfrFH3Q
GitHub repository (notebook, data and dashboard): https://github.com/pinological/LoRa_ST7072CEM_repo



(a) Roamer carried in a bag



(b) Roamer interface after a sweep

Fig. 24: Field use of the Roamer.

2. Load and combine the raw data

In [4]:
```
logs = []
for f in files:
    d = pd.read_csv(f)
    d['site'] = os.path.basename(f).split('_')[1]
    logs.append(d)
    print(f'{os.path.basename(f):26s} rows {len(d):5d}  ', d.status.value_counts().to_dict())
log = pd.concat(logs, ignore_index=True)
print('combined log:', log.shape)
log.groupby(['site', 'status']).size().unstack(fill_value=0)
```

```
site_A_roamer_log.csv      rows  1540  {'TIMEOUT': 967, 'OK': 563, 'NOACK': 10}
site_B_roamer_log.csv      rows  3262  {'OK': 1737, 'TIMEOUT': 1503, 'NOACK': 22}
site_C_roamer_log.csv      rows   376  {'OK': 197, 'TIMEOUT': 163, 'NOACK': 16}
site_D_roamer_log.csv      rows   368  {'TIMEOUT': 245, 'OK': 115, 'NOACK': 8}
combined log: (5546, 24)
```

Out[4]:

status	NOACK	OK	TIMEOUT
site			
A	10	563	967
B	22	1737	1503
C	16	197	163
D	8	115	245

In [5]:
```
sweeps = log[log.status.isin(['OK', 'TIMEOUT'])].copy()
sweeps['stop_key'] = sweeps.site + sweeps.stop_id.astype(int).astype(str)
slots = sweeps.groupby('stop_key').size()
complete = slots[slots == 90].index
raw = sweeps[sweeps.stop_key.isin(complete)].drop(columns='stop_key').reset_index(drop=True)
print('stops logged:', len(slots), '| complete stops:', len(complete), '| incomplete dropped:', int((slots != 90).sum()))

fails = (log[log.status == 'NOACK'][['site', 'stop_id', 'lat', 'lon']]
         .rename(columns={'stop_id': 'attempt_id'}).sort_values(['site', 'attempt_id']).reset_index(drop=True))
feat = pd.read_csv(os.path.join(DATA, 'stop_features.csv'))
stations = pd.read_csv(os.path.join(DATA, 'stations.csv'))
print('raw sweeps:', raw.shape, '| failed handshakes:', fails.shape, '| stop features:', feat.shape, '| stations:', stations.shape)
feat.head()
```

```
stops logged: 61 | complete stops: 61 | incomplete dropped: 0
raw sweeps: (5490, 24) | failed handshakes: (56, 4) | stop features: (61, 12) | stations: (4, 3)
```

Out[5]:

	stop_key	site	distance_to_station_m	station_ant_m	roamer_ant_m	stop_elev_m_srtm30	station_elev_m_srtm30	elev_diff_m	elev_angle_deg	terrain_category	land_cove
0	A13	A	236.0	2.0	1.3	1397	1372	25	5.88	mountain	fores
1	A14	A	202.0	2.0	1.3	1403	1372	31	8.53	mountain	mixe
2	A15	A	201.0	2.0	1.3	1403	1372	31	8.57	mountain	mixe
3	A16	A	182.0	2.0	1.3	1408	1372	36	10.98	mountain	fores
4	A18	A	229.0	2.0	1.3	1376	1372	4	0.83	mountain	mixe

3. Data checks

In [6]:
```
raw['stop_key'] = raw.site + raw.stop_id.astype(int).astype(str)
per_stop = raw.groupby('stop_key').size()
print('stops:', raw.stop_key.nunique(), '| every stop has 90 slots:', bool((per_stop == 90).all()))
print('slots:', len(raw), '| status counts:', raw.status.value_counts().to_dict())
print('received share: %.1f %%' % (100 * (raw.status == 'OK').mean()))
print('stops per site:', raw.groupby('site').stop_key.nunique().to_dict())
assert raw.stop_key.nunique() == 61 and (per_stop == 90).all() and len(raw) == 5490
assert set(raw.status.unique()) <= {'OK', 'TIMEOUT'}
assert raw.loc[raw.status == 'OK', 'rssi'].notna().all() and raw.loc[raw.status == 'TIMEOUT', 'rssi'].isna().all()
print('Cleaning checks passed: 61 complete stops, 5,490 slots, OK/TIMEOUT only, RSSI present exactly when OK.')
```

```
stops: 61 | every stop has 90 slots: True
slots: 5490 | status counts: {'TIMEOUT': 2878, 'OK': 2612}
received share: 47.6 %
stops per site: {'A': 17, 'B': 36, 'C': 4, 'D': 4}
Cleaning checks passed: 61 complete stops, 5,490 slots, OK/TIMEOUT only, RSSI present exactly when OK.
```

Fig. 25: Notebook: combining the site logs and data checks.

8. Path loss regression

In [15]:

```
from sklearn.linear_model import LinearRegression, RidgeCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import LeaveOneOut, LeaveOneGroupOut, cross_val_predict

y = d60.pl_median_14dbm.values; g = d60.case.values
MODELS = {
  'Mean only':                   (LinearRegression(), None),
  'Log-distance':                (LinearRegression(), ['logd']),
  '+ elev. diff. and angle':     (LinearRegression(), ['logd', 'elev_diff_m', 'elev_angle_deg']),
  '+ land cover (ridge)':        (make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 3, 30))), ['logd', 'elev_diff_m', 'elev_angle_deg'] +
  'Regression tree, depth 3':    (DecisionTreeRegressor(max_depth=3, min_samples_leaf=5, random_state=0), ['logd', 'elev_diff_m', 'elev_angle_deg'] +
rows = []
for name, (m, cols) in MODELS.items():
    X = np.zeros((len(y), 1)) if cols is None else d60[cols].values
    loo = cross_val_predict(m, X, y, cv=LeaveOneOut()); lc = cross_val_predict(m, X, y, groups=g, cv=LeaveOneGroupOut())
    rows.append([name, rmse(y, loo), rmse(y, lc)] + [rmse(y[g == c], lc[g == c]) for c in (1, 2, 3)])
reg = pd.DataFrame(rows, columns=['model', 'LOO', 'LOCO', 'LOCO case1', 'LOCO case2', 'LOCO case3']).round(1)
print(reg.to_string(index=False)); RESULTS.update(rmse_mean=reg.LOO[0], rmse_logd_loco=reg.LOCO[1], rmse_terrain_loco=reg.LOCO[2])

print('\nWithin one case (LOO inside the case):')
for c in (1, 2):
    dd = d60[d60.case == c]; yy = dd.pl_median_14dbm.values
    r = [rmse(yy, cross_val_predict(LinearRegression(), X_, yy, cv=LeaveOneOut())) for X_ in [np.zeros((len(yy), 1)), dd[['logd']], dd[['logd', 'elev_
    print(f'  Case {c} ({len(dd)} stops): mean {r[0]:.1f} | log-distance {r[1]:.1f} | + elevation diff {r[2]:.1f}')
d51 = d60[d60.n_ok >= 10]; y51 = d51.pl_median_14dbm.values
print('Strict subset (>=10 received, %d stops): mean %.1f | log-distance %.1f' % (len(d51),
      rmse(y51, cross_val_predict(LinearRegression(), np.zeros((len(y51), 1)), y51, cv=LeaveOneOut())),
      rmse(y51, cross_val_predict(LinearRegression(), d51[['logd']], y51, cv=LeaveOneOut()))))
print('Manual line-of-sight check, delivery %:', (s.groupby('line_of_sight').delivery_rate.mean() * 100).round(0).to_dict())

fig, ax = plt.subplots(figsize=(W, 2.4)); h = .36; yy_ = np.arange(len(reg))
ax.barh(yy_ - h/2, reg.LOO, h, color='#0072B2', label='Leave-one-stop-out'); ax.barh(yy_ + h/2, reg.LOCO, h, color='#E69F00', label='Leave-one-case-ou
for i, r in reg.iterrows():
    ax.text(r.LOO + .3, i - h/2, f'{r.LOO:.1f}', va='center', fontsize=7); ax.text(r.LOCO + .3, i + h/2, f'{r.LOCO:.1f}', va='center', fontsize=7)
ax.set_yticks(yy_); ax.set_yticklabels(reg.model); ax.invert_yaxis(); ax.set_xlim(0, 21)
ax.set_xlabel('RMSE of median path loss (dB), lower is better'); ax.grid(axis='y', visible=False)
ax.legend(fontsize=7, frameon=False, loc='lower center', bbox_to_anchor=(.4, 1.0), ncol=2)
fig.savefig(f'{FIG}/fig4_regression_rmse.png'); plt.show()
```

```
                 model  LOO  LOCO  LOCO case1  LOCO case2  LOCO case3
             Mean only 11.1  11.4         8.4        12.9         9.1
          Log-distance 11.3  13.0         9.8        14.8         9.5
+ elev. diff. and angle 11.4  15.8        11.2        16.3        20.9
   + land cover (ridge) 11.1  15.7         8.4        17.1        19.8
Regression tree, depth 3 11.2  15.3         9.5        16.9        16.8

Within one case (LOO inside the case):
  Case 1 (16 stops): mean 8.7 | log-distance 8.0 | + elevation diff 7.2
  Case 2 (36 stops): mean 12.6 | log-distance 12.9 | + elevation diff 11.9
Strict subset (>=10 received, 51 stops): mean 10.6 | log-distance 10.9
Manual line-of-sight check, delivery %: {'blocked': 27.0, 'clear': 76.0, 'partly blocked': 53.0}
```

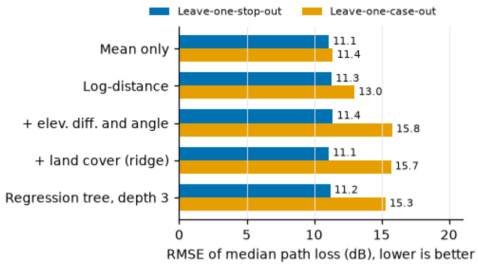


Fig. 26: Notebook: path loss regression.

Other models, metrics, ROC and confusion matrix

In [19]:

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (precision_score, recall_score, f1_score, confusion_matrix, roc_curve,
                             precision_recall_curve, average_precision_score)
SFP, ALLF = SETS['SF + power'], SETS['+ land cover']
mk_tree = lambda: DecisionTreeClassifier(max_depth=3, min_samples_leaf=90, random_state=0)
MODELS = {'Tree, SF + power': (mk_tree(), SFP), 'Tree, all features': (mk_tree(), ALLF),
          'Random forest, all features': (RandomForestClassifier(300, random_state=0, n_jobs=-1), ALLF),
          'Logistic regression, SF + power': (make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)), SFP),
          'Logistic regression, all features': (make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)), ALLF),
          'Gradient boosting, all features': (HistGradientBoostingClassifier(random_state=0), ALLF)}
PROBS = {'Link-budget rule': prob}; LOCO = {'Link-budget rule': np.nan}
for name, (mdl, cols) in MODELS.items():
    PROBS[name] = cross_val_predict(mdl, p[cols], yp, groups=GS, cv=gkf, method='predict_proba')[:, 1]
    LOCO[name] = balanced_accuracy_score(yp, cross_val_predict(mdl, p[cols], yp, groups=GC, cv=LeaveOneGroupOut(), method='predict_proba')[:, 1] > .5)
rows = []
for name, pr in PROBS.items():
    yhat = (pred > 0) if name == 'Link-budget rule' else (pr > .5)
    tn, fp, fn, tp = confusion_matrix(yp, yhat).ravel()
    rows.append([name, balanced_accuracy_score(yp, yhat), precision_score(yp, yhat), recall_score(yp, yhat), f1_score(yp, yhat),
                 roc_auc_score(yp, pr), average_precision_score(yp, pr), LOCO[name], tn, fp, fn, tp])
full = pd.DataFrame(rows, columns=['model', 'bal_acc', 'precision', 'recall', 'F1', 'AUC', 'AP', 'LOCO_bal_acc', 'TN', 'FP', 'FN', 'TP'])
print(full.round(3).to_string(index=False))
RESULTS.update(tree_precision=full.precision[1], tree_recall=full.recall[1], tree_f1=full.F1[1], gb_auc=full.AUC[6], gb_loco=full.LOCO_bal_acc[6],
               lr_sfp_loco=full.LOCO_bal_acc[4], linkbudget_recall=full.recall[0])

fig, ax = plt.subplots(1, 3, figsize=(W * 2.05, 2.45), gridspec_kw=dict(width_ratios=[1, 1, .82]))
for n, c, ls in [('Link-budget rule', '#999999', ':'), ('Tree, SF + power', '#0072B2', '-'),
                 ('Tree, all features', '#009E73', '--'), ('Random forest, all features', '#D55E00', '-.')]:
    fpr, tpr, _ = roc_curve(yp, PROBS[n]); ax[0].plot(fpr, tpr, c=c, ls=ls, lw=1.3, label=f'{n} ({roc_auc_score(yp, PROBS[n]):.2f})')
    pr_, rc_, _ = precision_recall_curve(yp, PROBS[n]); ax[1].plot(rc_, pr_, c=c, ls=ls, lw=1.3, label=f'{n} ({average_precision_score(yp, PROBS[n]):.
ax[0].plot([0, 1], [0, 1], c='#ccc', lw=.8); ax[0].set(xlabel='False positive rate', ylabel='True positive rate', title='(a) ROC (AUC)')
ax[1].axhline(yp.mean(), c='#ccc', lw=.8); ax[1].set(xlabel='Recall', ylabel='Precision', title='(b) Precision-recall (AP)', ylim=(.3, 1.02))
for a in ax[:2]: a.legend(fontsize=5.2, frameon=False, loc='lower right')
cm = confusion_matrix(yp, PROBS['Tree, SF + power'] > .5); ax[2].imshow(cm, cmap='Blues'); ax[2].grid(False)
for i in range(2):
    for j in range(2): ax[2].text(j, i, f'{cm[i, j]:,}', ha='center', va='center', fontweight='bold', color='white' if cm[i, j] > 1500 else 'black')
ax[2].set_xticks([0, 1], ['lost', 'delivered']); ax[2].set_yticks([0, 1], ['lost', 'delivered']); ax[2].set(xlabel='Predicted', ylabel='Actual', title
fig.tight_layout(); fig.savefig(f'{FIG}/fig9_roc_pr_cm.png'); plt.show()
```

model	bal_acc	precision	recall	F1	AUC	AP	LOCO_bal_acc	TN	FP	FN	TP
Link-budget rule	0.538	0.496	0.970	0.656	0.727	0.706	NaN	302	2576	78	2534
Tree, SF + power	0.679	0.699	0.587	0.638	0.713	0.684	0.671	2217	661	1078	1534
Tree, all features	0.620	0.625	0.528	0.573	0.641	0.600	0.501	2052	826	1233	1379
Random forest, all features	0.646	0.633	0.618	0.625	0.707	0.664	0.553	1942	936	998	1614
Logistic regression, SF + power	0.677	0.672	0.637	0.654	0.734	0.706	0.654	2066	812	949	1663
Logistic regression, all features	0.674	0.664	0.644	0.654	0.726	0.697	0.556	2026	852	929	1683
Gradient boosting, all features	0.680	0.661	0.674	0.667	0.745	0.723	0.576	1974	904	851	1761

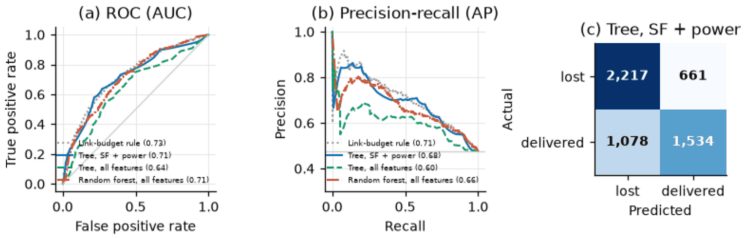


Fig. 27: Notebook: delivery classifiers, metrics, ROC and confusion matrix.

10. Cheapest setting per stop

In [20]:
```
mw = {2: 10 ** .2, 8: 10 ** .8, 14: 10 ** 1.4}
gs = p.groupby(['stop_key', 'case', 'sf', 'pwr']).ok.sum().reset_index()
gs['cost'] = 2.0 ** (gs.sf - 7) * gs.pwr.map(mw)
best = gs[gs.ok >= 4].sort_values('cost').groupby('stop_key').first()
ms = p.groupby('stop_key').case.first().to_frame().join(best[['sf', 'pwr', 'cost']])
full_cost = 32 * mw[14]
rows = []
for c, dd in list(ms.groupby('case')) + [('All', ms)]:
    sv = dd[dd.sf.notna()]
    rows.append([c, f'{len(sv)}/{len(dd)}', int(((sv.sf <= 9) & (sv.pwr <= 8)).sum()), int(((sv.sf >= 11) & (sv.pwr == 14)).sum()),
                 round(100 * (1 - sv.cost / full_cost).median())])
mt = pd.DataFrame(rows, columns=['case', 'stops with a working setting', 'low cost (SF<=9, <=8 dBm)', 'needed SF11-12 at 14 dBm', 'median energy savin
print(mt.to_string(index=False))
RESULTS.update(min_served=int(ms.sf.notna().sum()), min_saving=float(mt.iloc[-1, -1]))
```

```
case stops with a working setting  low cost (SF<=9, <=8 dBm)  needed SF11-12 at 14 dBm  median energy saving %
   1                       13/17                           4                          2                       97
   2                       34/36                          13                          3                       97
   3                         8/8                           2                          2                       92
 All                       55/61                          19                          7                       97
```

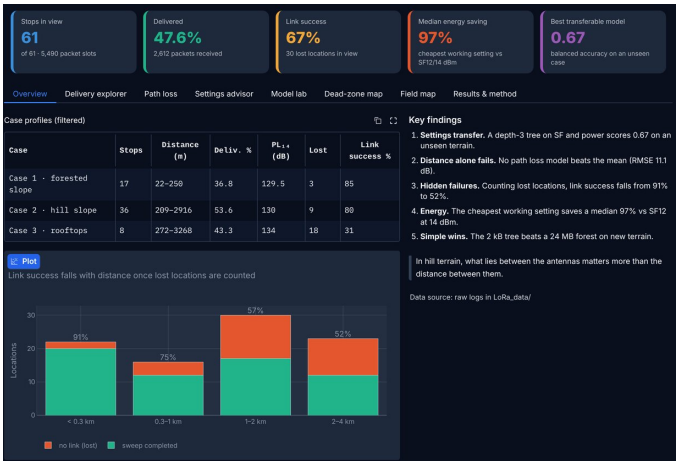
11. Computational cost

In [21]:
```
import time, pickle

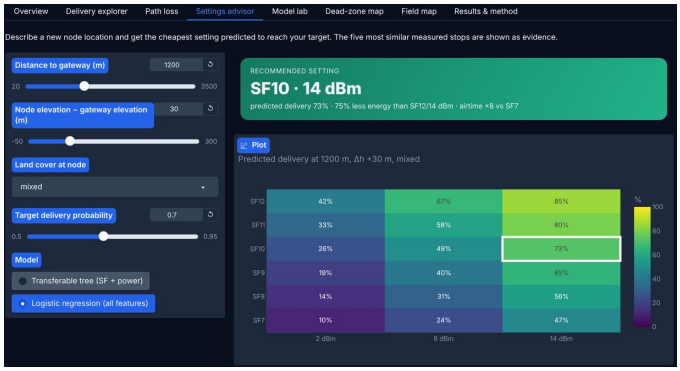
cols_r = ['logd', 'elev_diff_m', 'elev_angle_deg'] + LC
def cost(name, mdl, X, y_):
    ft = []; pt = []
    for _ in range(5):
        t0 = time.perf_counter(); mdl.fit(X, y_); ft.append(time.perf_counter() - t0)
        t0 = time.perf_counter(); mdl.predict(X); pt.append(time.perf_counter() - t0)
    return dict(model=name, fit_ms=1e3 * np.median(ft), predict_us_per_sample=1e6 * np.median(pt) / len(X), size_kB=len(pickle.dumps(mdl)) / 1024)
cc = pd.DataFrame([
    cost('Log-distance (OLS)', LinearRegression(), d60[['logd']], d60.pl_median_14dbm),
    cost('Ridge, all features', make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 3, 30))), d60[cols_r], d60.pl_median_14dbm),
    cost('Regression tree, depth 3', DecisionTreeRegressor(max_depth=3, min_samples_leaf=5, random_state=0), d60[cols_r], d60.pl_median_14dbm),
    cost('RF regressor, 300 trees', RandomForestRegressor(300, random_state=0, n_jobs=1), d60[cols_r], d60.pl_median_14dbm),
    cost('Tree, SF + power', mk_tree(), p[SFP], yp), cost('Tree, all features', mk_tree(), p[ALLF], yp),
    cost('RF classifier, 300 trees', RandomForestClassifier(300, random_state=0, n_jobs=1), p[ALLF], yp)]).round(2)
print(cc.to_string(index=False))
print('RF classifier is %.0fx larger than the SF + power tree' % (cc.size_kB.iloc[-1] / cc.size_kB.iloc[4]))
```

```
                model  fit_ms  predict_us_per_sample  size_kB
      Log-distance (OLS)    1.32                   9.68     0.59
     Ridge, all features    4.16                  13.92     1.42
Regression tree, depth 3    1.24                   9.72     1.83
 RF regressor, 300 trees  282.25                 431.54  1574.60
        Tree, SF + power    1.84                   0.13     2.40
      Tree, all features    2.97                   0.14     2.47
RF classifier, 300 trees  885.72                  14.62 24397.74
RF classifier is 10166x larger than the SF + power tree
```

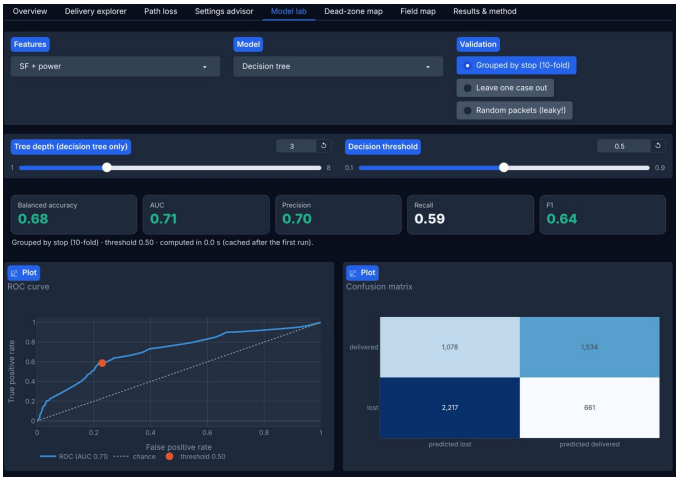
Fig. 28: Notebook: cheapest setting and computational cost.



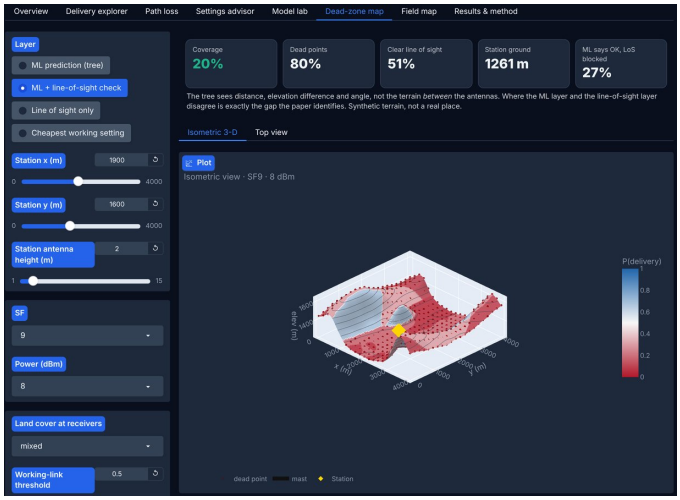
(a) Overview with filters and summary cards



(b) Settings advisor



(c) Model lab: ROC curve and confusion matrix



(d) Dead-zone map, isometric 3-D view

Fig. 29: Gradio dashboard, dark mode (no site names or coordinates).